THE SUMMIT PROJECTS

P5 Kubernetes

Project 5 of 9 | Uses levels: Level 8 | ~12 hours

1 Goal and outcome

Run `shop-api` on **Kubernetes**: the same container image your Project 3 pipeline publishes, the same `/healthz` and `/readyz` contract, the same Nginx-style request path, but scheduled, healed, scaled and rolled out by a cluster instead of by `docker compose` or a Terraform instance replacement.

Level 8's capstone Capstone 8 is the complete, topic-by-topic build with every manifest. This project is the integration step: it starts from that foundation, plugs in **your** image, ConfigMaps and Secrets from the earlier projects, and changes the deployment step in your pipeline from SSH to `kubectl`. Everything runs on a free local cluster (kind), so there is no cloud cost.

You finish with:

- a kind cluster with an Ingress controller and a `shop` namespace;
- PostgreSQL (StatefulSet with a PersistentVolumeClaim) and Redis (Deployment) in the cluster for the lab, and a clear note on where RDS from Project 4 would replace them;
- the API as a Deployment with two replicas, probes, resource requests and limits, a hardened security context, and a Service;
- `curl -H 'Host: shop.example.com' http://localhost:8080/products` working through the Ingress, with the cache header flipping from MISS to HIT;
- a rolling update that causes no failed requests, a rollback with one command, and a pipeline step `kubectl apply -k` that replaces `deploy.sh`.

2 Architecture

What changed from Project 4: the EC2 instances, user data and ALB target group are replaced by Kubernetes objects (marked NEW). Self-healing, rolling updates and service discovery move from scripts and cloud services into the cluster.

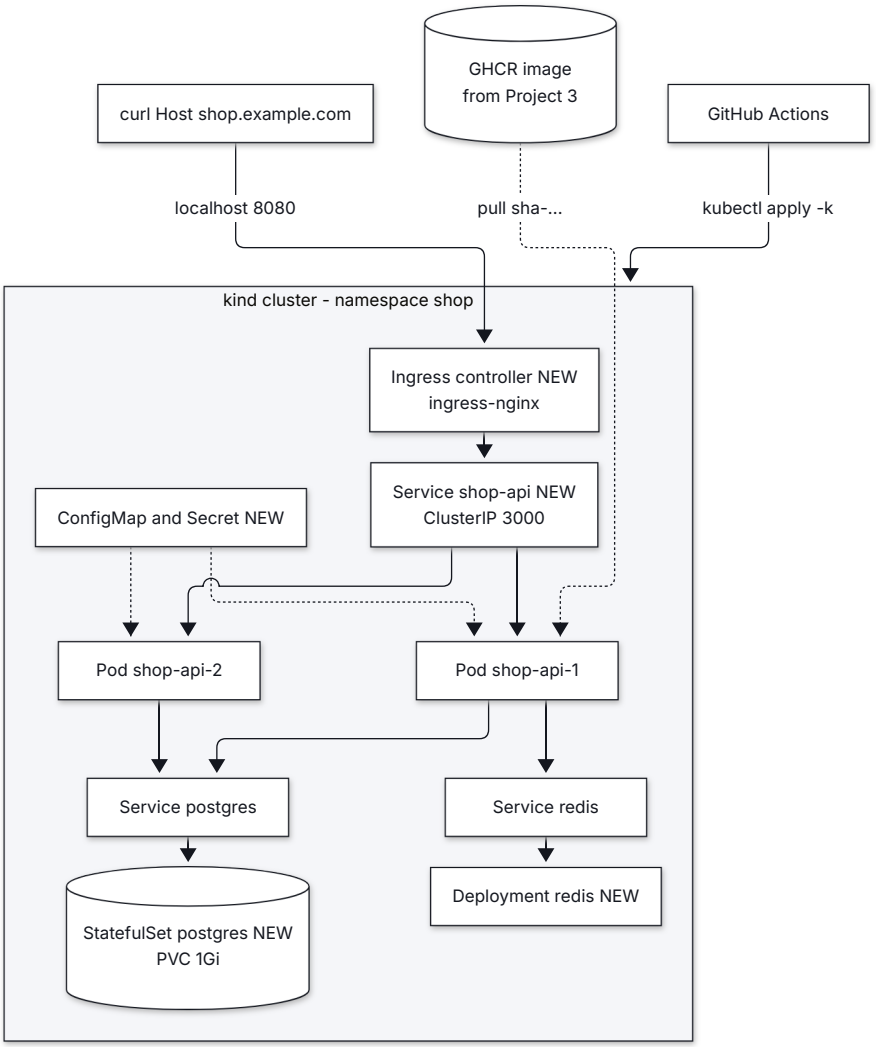


Figure 5-1

What happens when you roll out a new version:

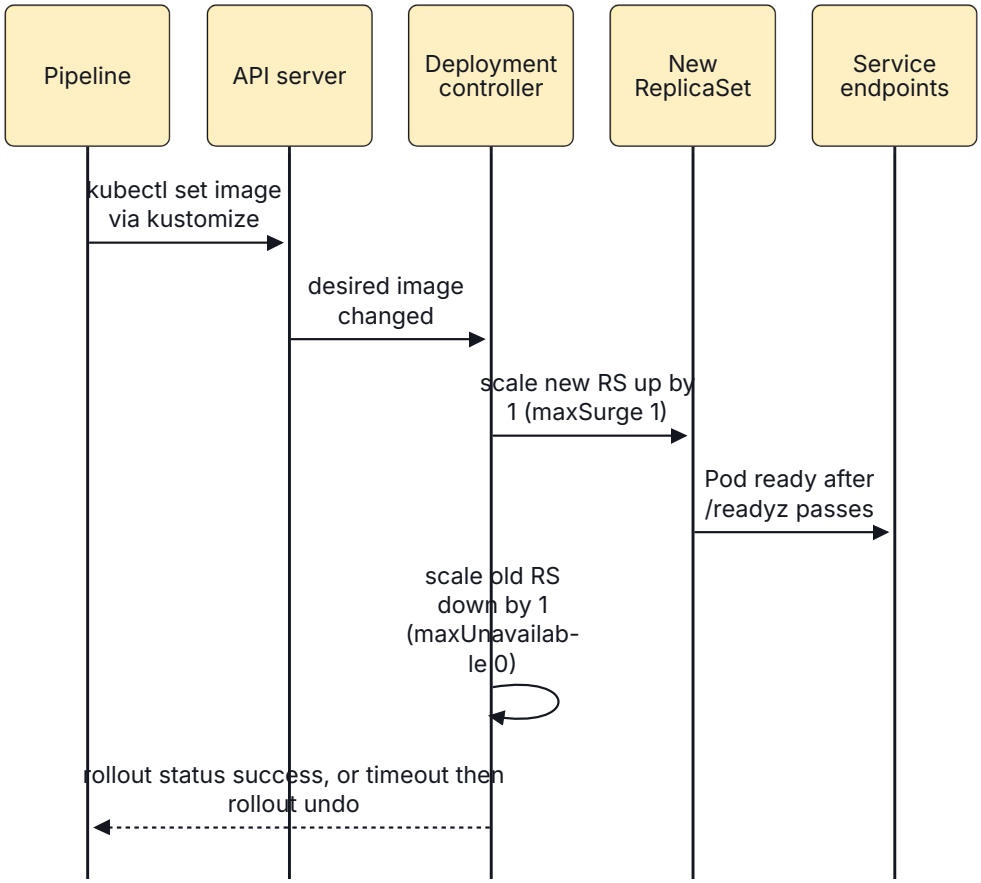


Figure 5-2

Project 4 (AWS)	Project 5 (Kubernetes)
2 EC2 instances, replaced on change	Deployment with rolling updates, `maxUnavailable: 0`
ALB health check `/readyz`	readiness probe `/readyz`, liveness probe `/healthz`
Security groups	NetworkPolicies (Project 6) and Pod security context
Secrets Manager plus instance role	Kubernetes Secret (hardened in Project 6)
`user_data` and env vars	ConfigMap and Secret
ASG or manual scaling	`replicas` now, HPA in Project 6
Terraform deploys the tag	`kustomize edit set image` plus `kubectl apply -k`

3 Prerequisites

- A laptop with Docker, kind (`kind.sigs.k8s.io`), `kubectl`, `curl`; about 8 GB RAM free. Cost: nothing. You may later repeat this on EKS using Project 4's VPC (cost: the EKS control plane and nodes bill hourly).
- The image `ghcr.io/<you>/shop-api:sha-<sha>` from Project 3 (public package, or `kind load docker-image` it).
- Kubernetes: cluster architecture §8.1, Pods §8.2, Deployments §8.3, Services §8.4, Ingress §8.5, ConfigMaps §8.6, Secrets §8.7, namespaces §8.8, volumes, PV and PVC §8.9, StatefulSets §8.10, requests and limits §8.14, probes §8.15, service discovery §8.17.
- Do not start here without comfortable Linux, networking and Docker skills. If `kubectl describe` output is hard to read, revisit Level 2 and Level 4 first.

4 Build it step by step

Work in a repository folder `k8s/` (in the `shop-api` repo) so that the manifests are versioned with the code.

Milestone 1 — Cluster and ingress controller (≈45 min)

Use the kind configuration and controller install from Milestone 1 of the Level 8 capstone Capstone 8 (a kind cluster with ports 80/443 mapped to `localhost:8080/8443`, a CNI that enforces NetworkPolicies such as Calico, and ingress-nginx). Note the capstone's caveat: ingress-nginx is retired upstream, so treat it as a learning tool and keep `ingressClassName` configurable. Verify:

```bash
kubectl get nodes                                  # all Ready
kubectl -n ingress-nginx get pods                  # controller Running
```

Milestone 2 — Namespace, configuration and secrets (≈45 min)

`k8s/namespace.yaml`:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/warn: restricted
```

`k8s/configmap.yaml`:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-api-config
  namespace: shop
data:
  PORT: "3000"
  HOST: "0.0.0.0"
  REDIS_URL: "redis://redis:6379"
```

Secrets must not be committed. Create the database credentials imperatively (Project 6 replaces this with a proper secret workflow):

```bash
kubectl apply -f k8s/namespace.yaml -f k8s/configmap.yaml
DB_PASS="$(openssl rand -hex 16)"
kubectl -n shop create secret generic shop-db \
  --from-literal=POSTGRES_USER=shop --from-literal=POSTGRES_PASSWORD="$DB_PASS" \
  --from-literal=DATABASE_URL="postgres://shop:${DB_PASS}@postgres:5432/shop"
kubectl -n shop get secret shop-db -o jsonpath='{.data.POSTGRES_USER}' | base64 -d     # shop
```

Remember that base64 is encoding, not encryption; anyone who can `get secrets` in the namespace can read them.

Milestone 3 — Data tier: PostgreSQL and Redis (≈1.5 h)

`k8s/postgres.yaml` (a headless Service for stable DNS, a StatefulSet with a volume claim):

```yaml
apiVersion: v1
kind: Service
metadata: { name: postgres, namespace: shop }
spec:
  clusterIP: None
  selector: { app: postgres }
  ports: [{ name: pg, port: 5432 }]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: postgres, namespace: shop }
spec:
  serviceName: postgres
  replicas: 1
  selector: { matchLabels: { app: postgres } }
```

```
  template:
    metadata: { labels: { app: postgres } }
    spec:
      securityContext: { runAsUser: 70, runAsGroup: 70, fsGroup: 70 }
      containers:
        - name: postgres
          image: postgres:17-alpine
          ports: [{ containerPort: 5432, name: pg }]
          env:
            - { name: POSTGRES_DB, value: shop }
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          envFrom: [{ secretRef: { name: shop-db } }]
          readinessProbe:
            exec: { command: ["pg_isready", "-U", "shop", "-d", "shop"] }
            periodSeconds: 5
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits: { memory: 512Mi }
          volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
  volumeClaimTemplates:
    - metadata: { name: data }
      spec:
        accessModes: ["ReadWriteOnce"]
        resources: { requests: { storage: 1Gi } }
```

`k8s/redis.yaml` (cache only: no persistence, so a Deployment is enough):

YAML

```yaml
apiVersion: v1
kind: Service
metadata: { name: redis, namespace: shop }
spec:
  selector: { app: redis }
  ports: [{ port: 6379 }]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: redis, namespace: shop }
spec:
  replicas: 1
  selector: { matchLabels: { app: redis } }
  template:
    metadata: { labels: { app: redis } }
    spec:
      containers:
        - name: redis
          image: redis:7.4-alpine
          args: ["--save", "", "--appendonly", "no", "--maxmemory", "64mb", "--maxmemory-
  policy", "allkeys-lru"]
          ports: [{ containerPort: 6379 }]
          readinessProbe: { exec: { command: ["redis-cli", "ping"] }, periodSeconds: 5 }
```

```
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits: { memory: 128Mi }
```

Seed the schema with a one-off Job (the `db/init.sql` from Project 2 in a ConfigMap), and make it idempotent:

```bash
kubectl -n shop create configmap shop-init-sql --from-file=init.sql=db/init.sql
```

```yaml
apiVersion: batch/v1
kind: Job
metadata: { name: shop-seed, namespace: shop }
spec:
  backoffLimit: 4
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: psql
          image: postgres:17-alpine
          command: ["sh", "-c", "until pg_isready -h postgres -U shop; do sleep 2; done;
  psql \"$DATABASE_URL\" -f /sql/init.sql"]
          envFrom: [{ secretRef: { name: shop-db } }]
          volumeMounts: [{ name: sql, mountPath: /sql }]
      volumes:
        - { name: sql, configMap: { name: shop-init-sql } }
```

`init.sql` uses `CREATE TABLE IF NOT EXISTS`, but `INSERT` would duplicate rows on re-run; guard it with `ON CONFLICT DO NOTHING` and a unique key on `name`, or seed only when the table is empty.

Milestone 4 — The API: Deployment and Service (≈1.5 h)

`k8s/api.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata: { name: shop-api, namespace: shop }
spec:
  selector: { app: shop-api }
  ports: [{ name: http, port: 3000, targetPort: http }]
---
apiVersion: apps/v1
kind: Deployment
```

```
metadata: { name: shop-api, namespace: shop }
spec:
  replicas: 2
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  selector: { matchLabels: { app: shop-api } }
  template:
    metadata: { labels: { app: shop-api } }
    spec:
      terminationGracePeriodSeconds: 30
      securityContext: { runAsNonRoot: true, seccompProfile: { type: RuntimeDefault } }
      containers:
        - name: api
          image: ghcr.io/you/shop-api:sha-REPLACED_BY_PIPELINE
          ports: [{ name: http, containerPort: 3000 }]
          envFrom:
            - configMapRef: { name: shop-api-config }
            - secretRef: { name: shop-db }
          startupProbe:   { httpGet: { path: /healthz, port: http }, periodSeconds: 2,
  failureThreshold: 30 }
          livenessProbe:  { httpGet: { path: /healthz, port: http }, periodSeconds: 10,
  failureThreshold: 3 }
          readinessProbe: { httpGet: { path: /readyz,  port: http }, periodSeconds: 5,
  failureThreshold: 2 }
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { memory: 256Mi }
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: ["ALL"] }
```

Two details matter. `/readyz` checks PostgreSQL, so a Pod whose database is down is removed from the Service without being restarted, whereas `/healthz` has no dependency checks, so a database outage does not cause a restart storm (probes §8.15). And the container exits cleanly on `SIGTERM`, which is what lets `maxUnavailable: 0` mean zero failed requests.

Milestone 5 — Ingress and Kustomize (≈45 min)

`k8s/ingress.yaml`:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: shop-api, namespace: shop }
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - { path: /, pathType: Prefix, backend: { service: { name: shop-api, port: {
  name: http } } } }
```

`k8s/kustomization.yaml` (so the pipeline can set the image tag without editing YAML by hand):

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop
resources: [api.yaml, ingress.yaml, postgres.yaml, redis.yaml]
images:
  - name: ghcr.io/you/shop-api
    newTag: sha-REPLACED_BY_PIPELINE
```

```bash
cd k8s && kubectl kustomize . >/dev/null && echo "renders"
kustomize edit set image ghcr.io/you/shop-api=ghcr.io/you/shop-api:sha-<sha>      # or:
  kubectl kustomize edit set image
kubectl apply -k .
kubectl -n shop rollout status deploy/shop-api --timeout=120s
curl -si -H 'Host: shop.example.com' http://localhost:8080/products | grep -iE 'HTTP/|x-
  cache'
```

Milestone 6 — Replace `deploy.sh` in the pipeline (≈1.5 h)

In the `deploy` job of Project 3, replace the SSH step with a cluster step. For a local kind cluster, run a **self-hosted runner** on your machine or deploy manually; for a cloud cluster use OIDC to get short-lived credentials (EKS access entries):

```yaml
      - name: Deploy to Kubernetes
        run: |
          cd k8s
          kustomize edit set image ghcr.io/${{ github.repository }}=ghcr.io/${{
  github.repository }}:sha-${{ github.sha }}
          kubectl apply -k .
          kubectl -n shop rollout status deploy/shop-api --timeout=180s || {
            kubectl -n shop rollout undo deploy/shop-api; exit 1; }
      - name: Smoke test
        run: BASE=https://shop.example.com ./smoke.sh
```

A running cluster reachable by a hosted runner needs credentials and network access; do not expose a lab API server to the internet for this. This is the reason Project 6 moves to a pull-based model (Argo CD): the cluster pulls changes, and CI needs no cluster credentials.

5 Tasks

- [ ] **T1 — Create the cluster and inspect it** *(≈45 min, beginner)*
  - **Goal:** know what is running before you add anything.
  - **Steps:**
    1. Create the kind cluster and install the Ingress controller.
    2. Run `kubectl get pods -A`, `kubectl get nodes -o wide`, and find the API server, etcd, scheduler and controller-manager Pods in `kube-system`.
    3. Sketch the control plane and node components from memory and compare with cluster architecture §8.1.
  - **Done when:** all nodes are `Ready` and you can name which component created each Pod you see.
- [ ] **T2 — Deploy the data tier** *(≈1 h, intermediate)*
  - **Goal:** PostgreSQL with durable storage and Redis for caching.
  - **Steps:**
    1. Apply the namespace, ConfigMap, Secret, `postgres.yaml` and `redis.yaml`.
    2. Run the seed Job and read its logs.
    3. Delete the Pod `postgres-0` and wait for it to return; query the table again.

- **Done when:** the three products are still present after the Pod is recreated, and `kubectl -n shop get pvc` shows `Bound`.

- [ ] **T3 — Run the API with probes and limits** *(≈1 h, intermediate)*
  - **Goal:** two healthy replicas behind a Service.
  - **Steps:**
    1. Apply `api.yaml` with your Project 3 image tag.
    2. `kubectl -n shop get endpoints shop-api` and `kubectl -n shop describe pod -l app=shop-api`.
    3. `kubectl -n shop port-forward svc/shop-api 3000:3000` and curl `/readyz`.
  - **Done when:** two Pods are `Ready`, the Service lists two endpoint addresses, and `/readyz` returns 200.

- [ ] **T4 — Expose it through Ingress** *(≈30 min, intermediate)*
  - **Goal:** the same URL path you had with Nginx, now through an Ingress.
  - **Steps:**
    1. Apply `ingress.yaml`.
    2. `curl -si -H 'Host: shop.example.com' http://localhost:8080/products` twice.
    3. Run `kubectl -n shop logs deploy/shop-api` and `kubectl -n ingress-nginx logs deploy/ingress-nginx-controller` to follow one request.
  - **Done when:** the first call has `X-Cache: MISS`, the second `HIT`, and you found your request in both logs.

- [ ] **T5 — Roll out a change with zero failed requests** *(≈45 min, intermediate)*
  - **Goal:** prove `maxUnavailable: 0` works.
  - **Steps:**
    1. In one terminal run `while true; do curl -s -o /dev/null -w '%{http_code}\n' -H 'Host: shop.example.com' http://localhost:8080/readyz; sleep 0.2; done | sort | uniq -c`.
    2. In another, change the image tag and `kubectl apply -k .`.
    3. Stop the loop and read the counts. Then repeat after changing the strategy to `maxUnavailable: 1` and `terminationGracePeriodSeconds: 0`.
  - **Done when:** the first rollout shows only `200`, and you can explain any non-200 in the second.

- [ ] **T6 — Break it and recover with rollback** *(≈45 min, intermediate)*
  - **Goal:** a bad release does not take the site down.

- **Steps:**
  1. Set the image to a tag that does not exist and apply. Watch `kubectl -n shop get pods -w` (`ImagePullBackOff`).
  2. Set `readinessProbe.path` to `/nope` and apply (the new Pods never become Ready).
  3. Use `kubectl -n shop rollout undo deploy/shop-api` and `kubectl -n shop rollout history deploy/shop-api`.
- **Done when:** in both cases the old Pods kept serving traffic during the failed rollout, and `rollout undo` restored the working version.

- [ ] **T7 — Wire the pipeline to the cluster** *(≈1.5 h, advanced)*
  - **Goal:** merge to `main` deploys to Kubernetes.
  - **Steps:**
    1. Install a self-hosted runner on the machine that runs kind, or script the deploy locally with `kustomize edit set image` as above.
    2. Replace the SSH deploy step with the Kubernetes step and add the automatic `rollout undo`.
    3. Merge a visible change and watch the run.
  - **Done when:** the commit's `sha-...` image is running in the cluster (`kubectl -n shop get deploy shop-api -o jsonpath='{.spec.template.spec.containers[0].image}'`) and the smoke test passed.

- [ ] **T8 — Prove it: explain a request and a failure** *(≈1 h, advanced)*
  - **Goal:** show the cluster is understood, not just running.
  - **Steps:**
    1. Without the guide, write down every object and DNS name touched by `curl -H 'Host: shop.example.com' http://localhost:8080/products`.
    2. Delete a node's Pod, then scale Redis to 0, and describe what each user sees and why.
    3. Compare Project 4 (EC2/ALB) and Project 5 on updates, health, secrets and scaling in a table of at least five rows.
  - **Done when:** your trace and table are in `docs/project-05.md`, and both failure behaviors match your predictions.

6 Definition of done

- [ ] `kubectl -n shop get pods` shows API (2), Redis (1) and PostgreSQL (1) Pods `Running` and `Ready`.

- [ ] `curl -H 'Host: shop.example.com' http://localhost:8080/products` works through the Ingress, with MISS then HIT.
- [ ] The API Deployment has requests and limits, startup, liveness and readiness probes, non-root user, read-only root file system and dropped capabilities.
- [ ] A rolling update shows no failed requests, and `kubectl rollout undo` recovers a bad release.
- [ ] The database survives `kubectl delete pod postgres-0`.
- [ ] No secret value is committed; manifests contain only references.
- [ ] The pipeline (or a documented command) deploys by image tag, never `latest`.

7 Break it on purpose

1. **Limit too low.** Set the API memory limit to `32Mi`. Watch `OOMKilled` and `CrashLoopBackOff` in `kubectl describe pod`, find the last state and exit code 137, and raise the limit. *(Lesson: requests schedule Pods, limits kill them.)*
2. **Selector mismatch.** Change the Service selector to `app: shop-apii`. The Service has no endpoints and the Ingress returns 503. Find it with `kubectl get endpoints`. *(Lesson: Services and Pods are linked only by labels.)*
3. **Break DNS naming.** Recreate the Secret with a `DATABASE_URL` whose host is `postgres.default.svc.cluster.local` (the wrong namespace) and restart the Deployment. Use a debug Pod (`kubectl run -it --rm debug --image=busybox:1.37 -n shop -- nslookup postgres`) to diagnose why `/readyz` fails. *(Lesson: Service DNS names include the namespace.)*
4. **Lose the database Pod's volume claim.** Delete the PVC while the Pod is running and see why it stays `Terminating` until the Pod is gone (protection finalizer); then recreate the lab and restore from `pg_dump`. *(Lesson: Kubernetes protects data objects, but not from every mistake.)*

8 How this evolves into the next project

This cluster works, but it is a **demo-grade** setup: no TLS, plaintext secrets, any Pod can talk to any Pod, nobody is limited by RBAC, the replicas are fixed, manifests are copy-edited per environment, and CI holds credentials for the cluster. Project 6 — Production Kubernetes closes those gaps: TLS on the Ingress, an HPA, RBAC and NetworkPolicies, a proper secrets workflow, a Helm chart replacing the raw YAML, and Argo CD pulling from Git so that CI no longer needs cluster credentials.